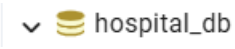
Problem Statement

Hospital Staff Management System

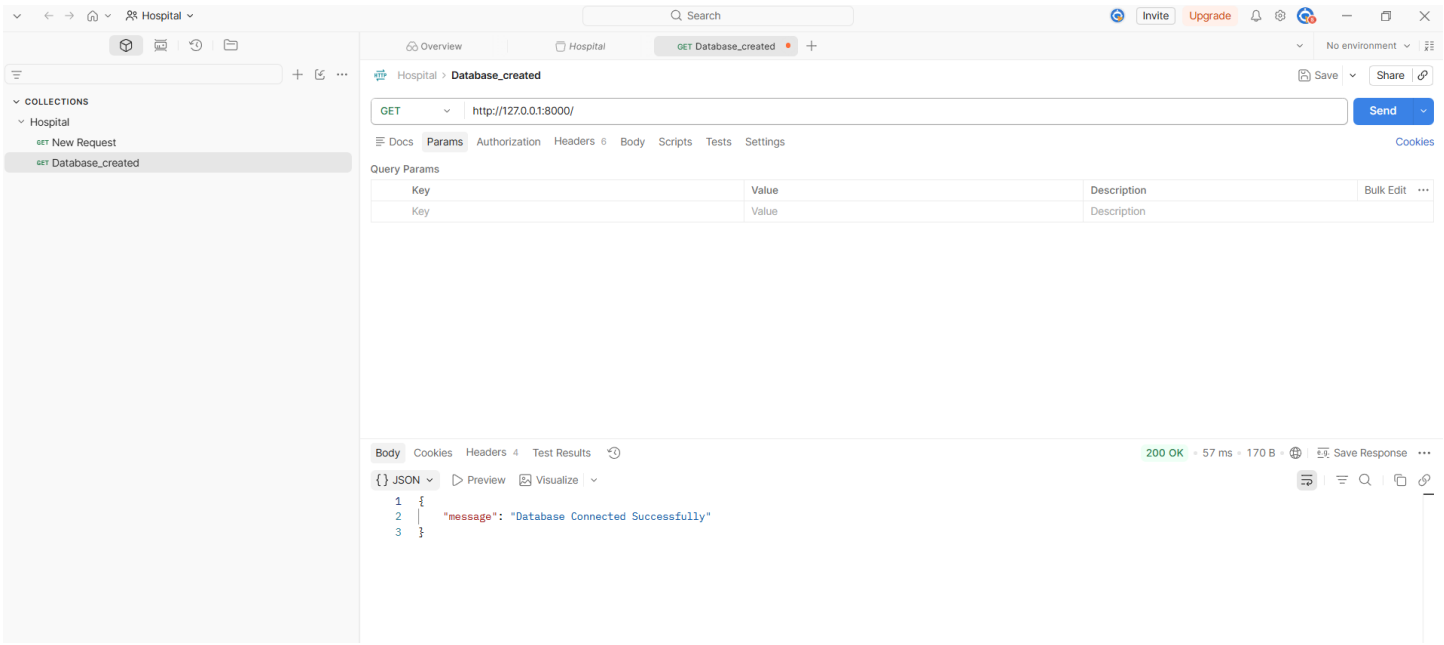
Tasks to Perform:

Task 1: Create PostgreSQL database.

PostgreSQL:

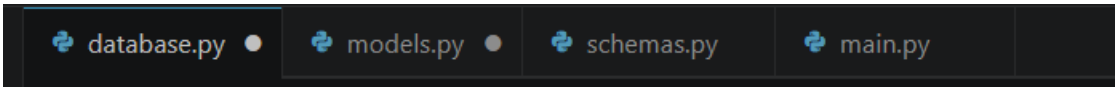


Postman:



Task 2: Create:

- **database.py**
- **models.py**
- **schemas.py**
- **main.py**

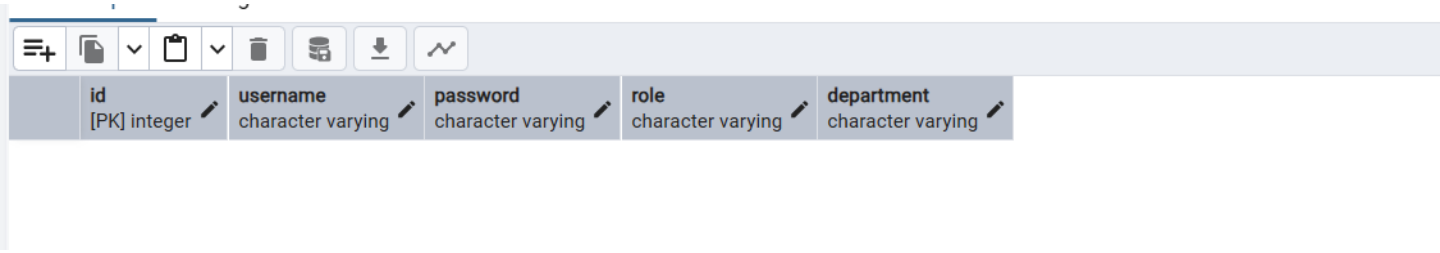


Task 3: Create SQLAlchemy model:

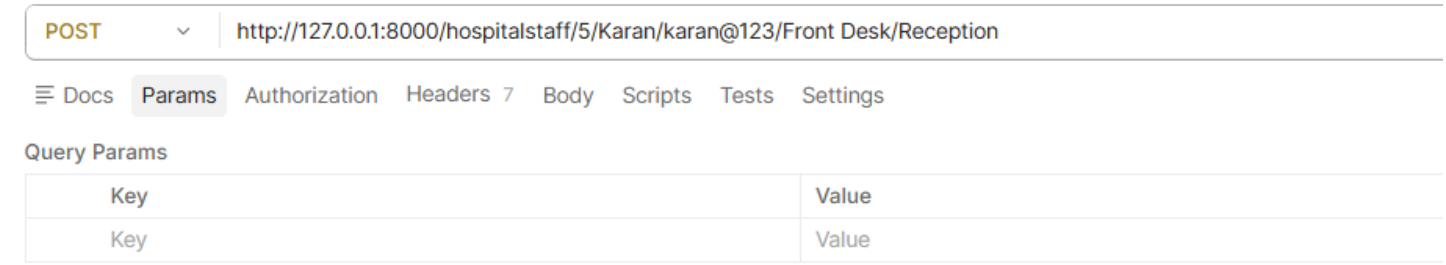
database.py | models.py | schemas.py | main.py

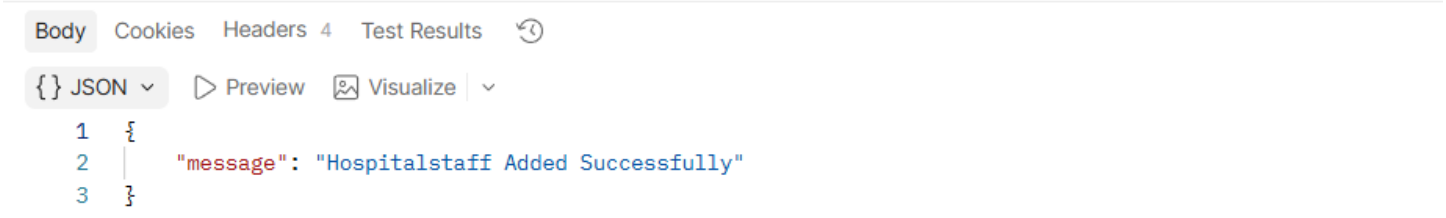
models.py > HospitalStaff

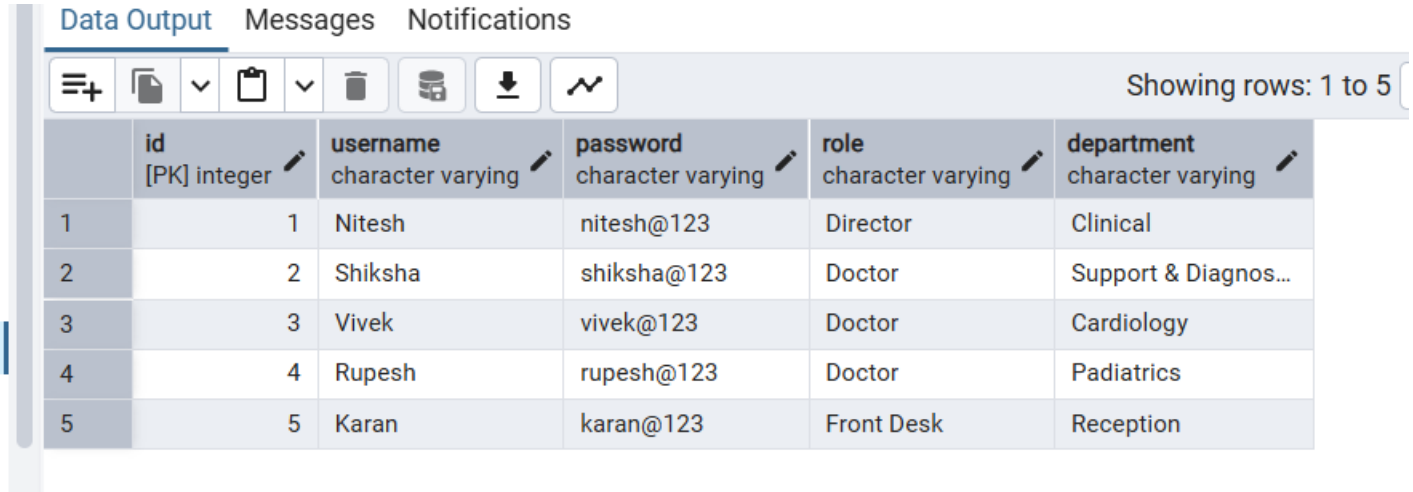
```python
from sqlalchemy import Column, Integer, String
# It is just a table design

from database import Base

class HospitalStaff(Base):

# class used to define structure of table

    __tablename__ = "HospitalStaff"

    id = Column(Integer, primary_key= True)

    username = Column(String)

    password = Column(String)

    role = Column(String)

    department = Column(String)


```



	id [PK] integer	username character varying	password character varying	role character varying	department character varying







	id [PK] integer	username character varying	password character varying	role character varying	department character varying
1	1	Nitesh	nitesh@123	Director	Clinical
2	2	Shiksha	shiksha@123	Doctor	Support & Diagnos...
3	3	Vivek	vivek@123	Doctor	Cardiology
4	4	Rupesh	rupesh@123	Doctor	Padiatrics
5	5	Karan	karan@123	Front Desk	Reception

Task 4: Create Pydantic Models for:

- **User Registration**
- **User Login**

- **User Registration**
- **User Login**

Schemas:

```python
from pydantic import BaseModel
# Basemodel, is used to define the structure of data and validate input data
```

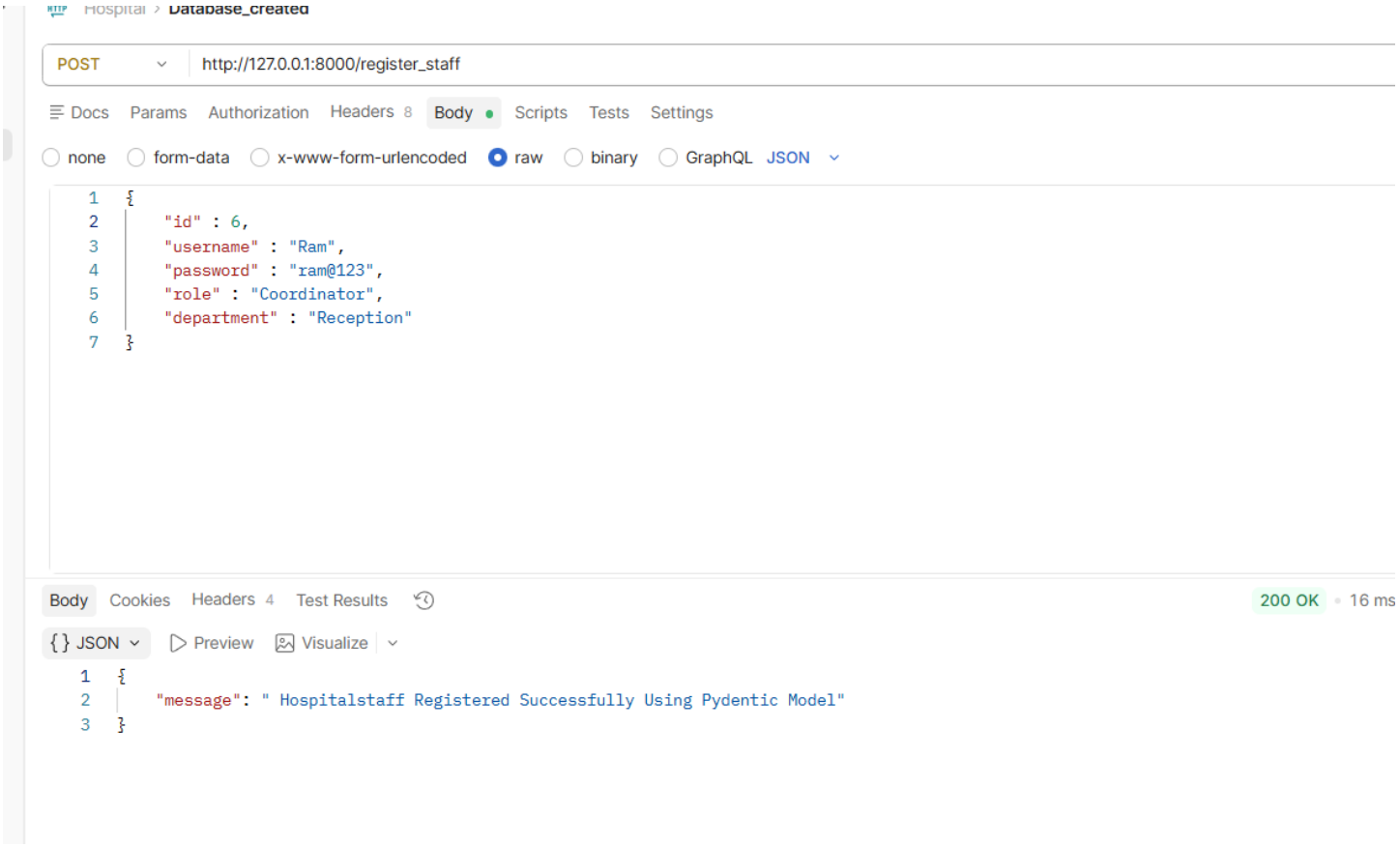
```python
class LoginSchema(BaseModel):

    username : str

    password : str
```

Main:

```python
@app.post("/register_staff")

def add_hospitalstaff(stf: HospitalStaffSchema):
    # staff must follow the structure defined in HospitalStaffSchema
    # i.e. it accepts JSON data and validate it using Pydantic model

    hospitalstaff = HospitalStaff(id = stf.id, username = stf.username, password = stf.password, role = stf.role, department = stf.
    # access Hospitalstaff table, find hospitalstaff whose ID matches, and return first matching row

    session.add(hospitalstaff)

    session.commit()

    return{"message": " Hospitalstaff Registered Successfully Using Pydentic Model"}

# get api

@app.get("/staff_login")

def get_hospitalstaff():

    hospitalstaff = session.query(HospitalStaff).all()

    return hospitalstaff
```

Task 5: Create User Registration API Requirements:

- **Register new staff members**
- **Store username, password, role and department**



	id [PK] integer	username character varying	password character varying	role character varying	department character varying
1	1	Nitesh	nitesh@123	Director	Clinical
2	2	Shiksha	shiksha@123	Doctor	Support & Diagnos...
3	3	Vivek	vivek@123	Doctor	Cardiology
4	4	Rupesh	rupesh@123	Doctor	Padiatrics
5	5	Karan	karan@123	Front Desk	Reception
6	6	Ram	ram@123	Coordinator	Reception
7	7	Rohan	rohan@123	Doctor	Orthopedics

Task 6: Implement Password Hashing Requirements:

- **Hash password before storing it in the database**
- **Use bcrypt hashing**

POST http://127.0.0.1:8000/register_hashed

Params Authorization Headers 8 Body Scripts Settings

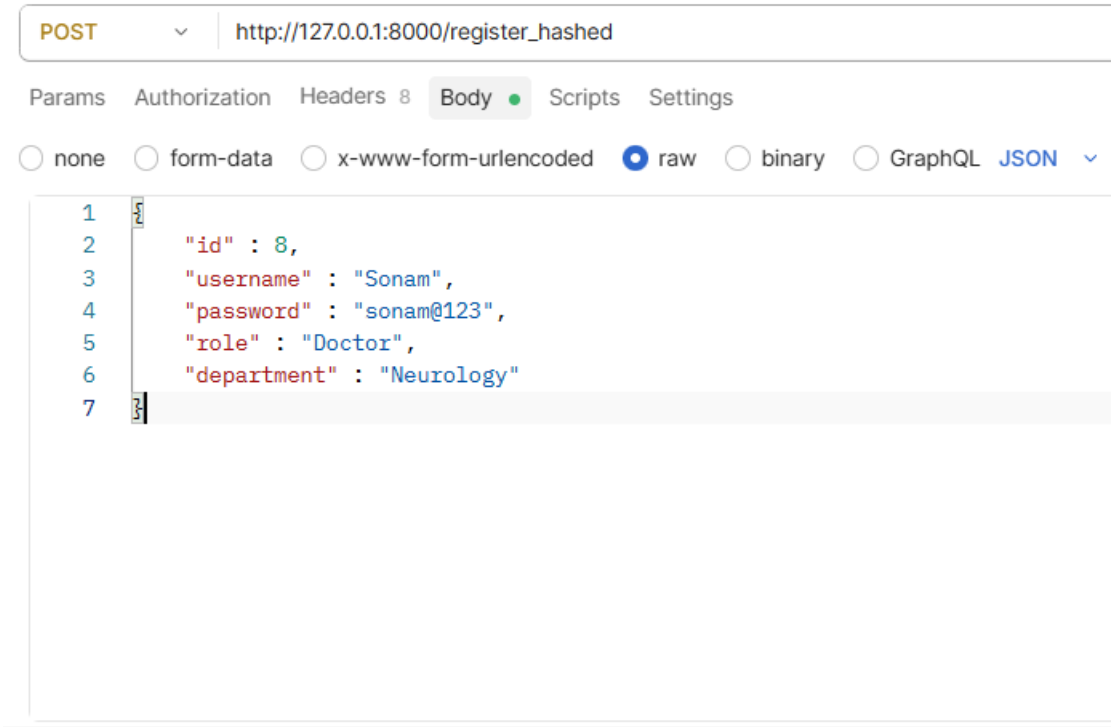
none form-data x-www-form-urlencoded raw binary GraphQL JSON

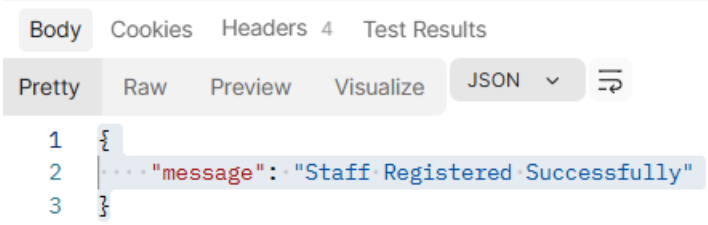
```
{
    "id" : 6,
    "username" : "Ram",
    "password" : "ram@123",
    "role" : "Coordinator",
    "department" : "Reception"
}
```

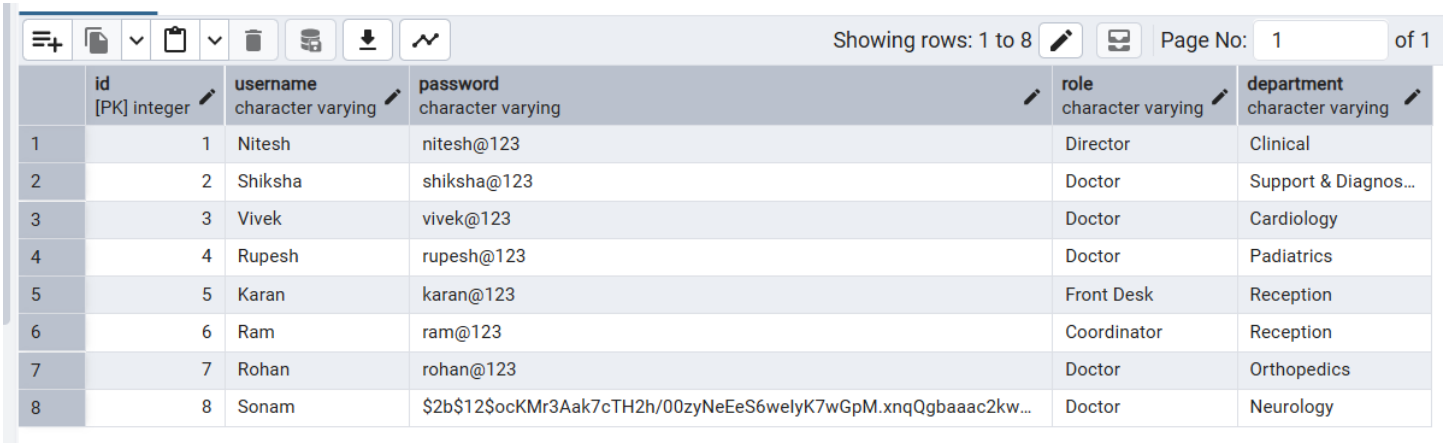
Body Cookies Headers 4 Test Results

Pretty Raw Preview Visualize JSON

```
{
    "detail": "Username already exists"
}
```





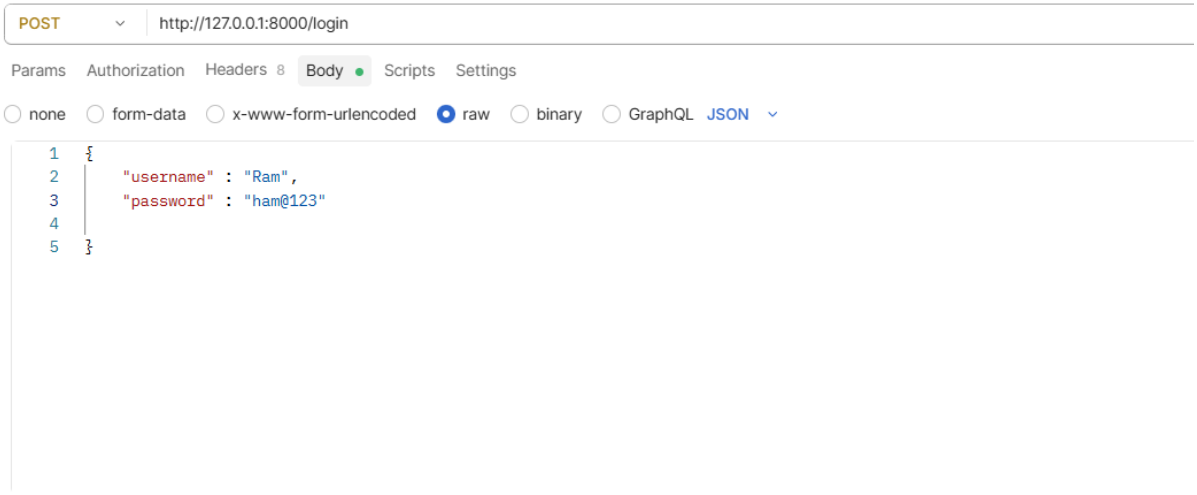


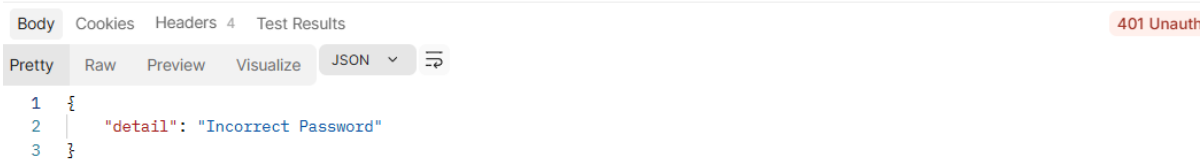
Showing rows: 1 to 8 Page No: 1 of 1

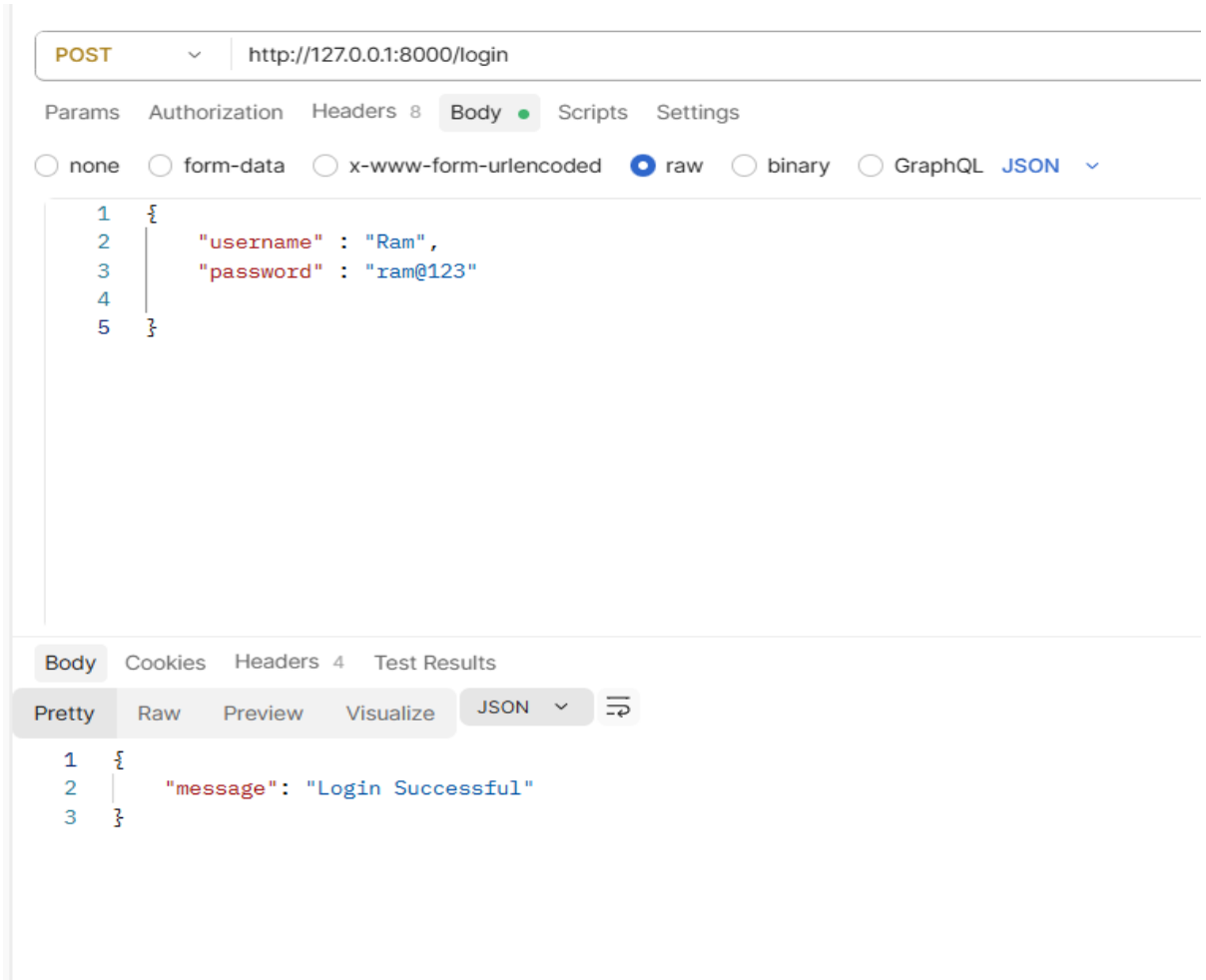
	id [PK] integer	username character varying	password character varying	role character varying	department character varying
1	1	Nitesh	nitesh@123	Director	Clinical
2	2	Shiksha	shiksha@123	Doctor	Support & Diagnos...
3	3	Vivek	vivek@123	Doctor	Cardiology
4	4	Rupesh	rupesh@123	Doctor	Padiatrics
5	5	Karan	karan@123	Front Desk	Reception
6	6	Ram	ram@123	Coordinator	Reception
7	7	Rohan	rohan@123	Doctor	Orthopedics
8	8	Sonam	$2b$12$ocKMr3Aak7cTH2h/00zyNeEeS6welyK7wGpM.xnqQgbaaac2kw...	Doctor	Neurology

Task 7: Create Login API Requirements:

- **Verify username**
- **Verify password**


POST
http://127.0.0.1:8000/login
Params
Authorization
Headers 8
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"username" : "Ram",
"password" : "ham@123"
}


Body
Cookies
Headers 4
Test Results
401 Unauth
Pretty
Raw
Preview
Visualize
JSON
{
"detail": "Incorrect Password"
}


POST
http://127.0.0.1:8000/login
Params
Authorization
Headers 8
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"username" : "Ram",
"password" : "ram@123"
}
Body
Cookies
Headers 4
Test Results
Pretty
Raw
Preview
Visualize
JSON
{
"message": "Login Successful"
}

• **Generate JWT token after successful login**

```
    token = create_token({"username":user.username})

# create a JWT token and store the username inside the token payload
# the username is stored in the token so that user does not need to login for every request

    return {"access_token": token}
# return the generated token to the user after successful login
```

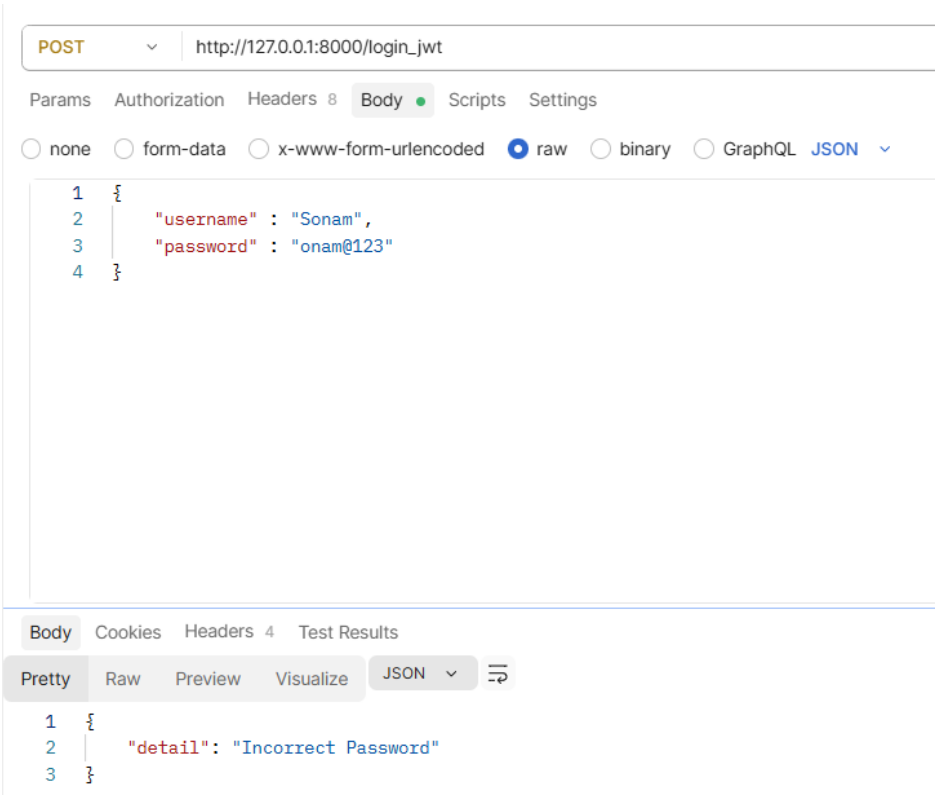
Task 8: Implement JWT Authentication Requirements:

• **Create JWT token generation function**

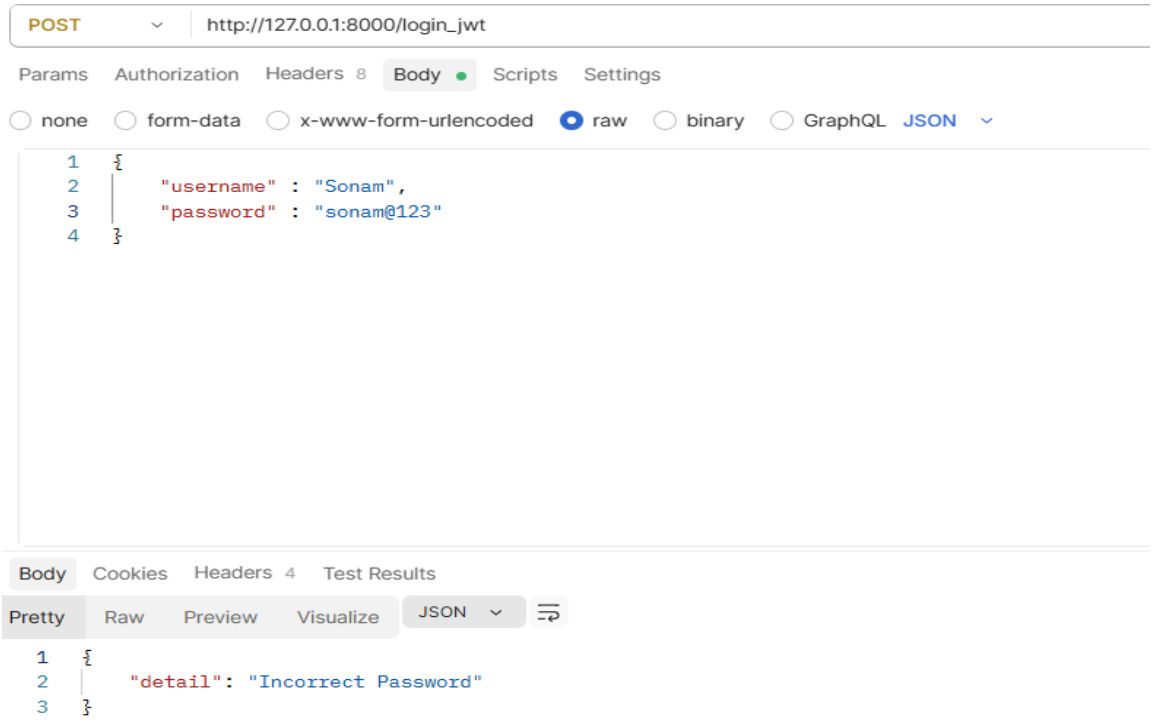
• **Create JWT verification function**

```
from jose import jwt

SECRET_KEY = "mysecretkey"
# secret key is used to create and verify JWT tokens
# "mysecretkey" is the secret password  used to sign and verify JWT tokens.

ALGORITHM = "HS256"
# a method used to encrypt/ sign the token
```

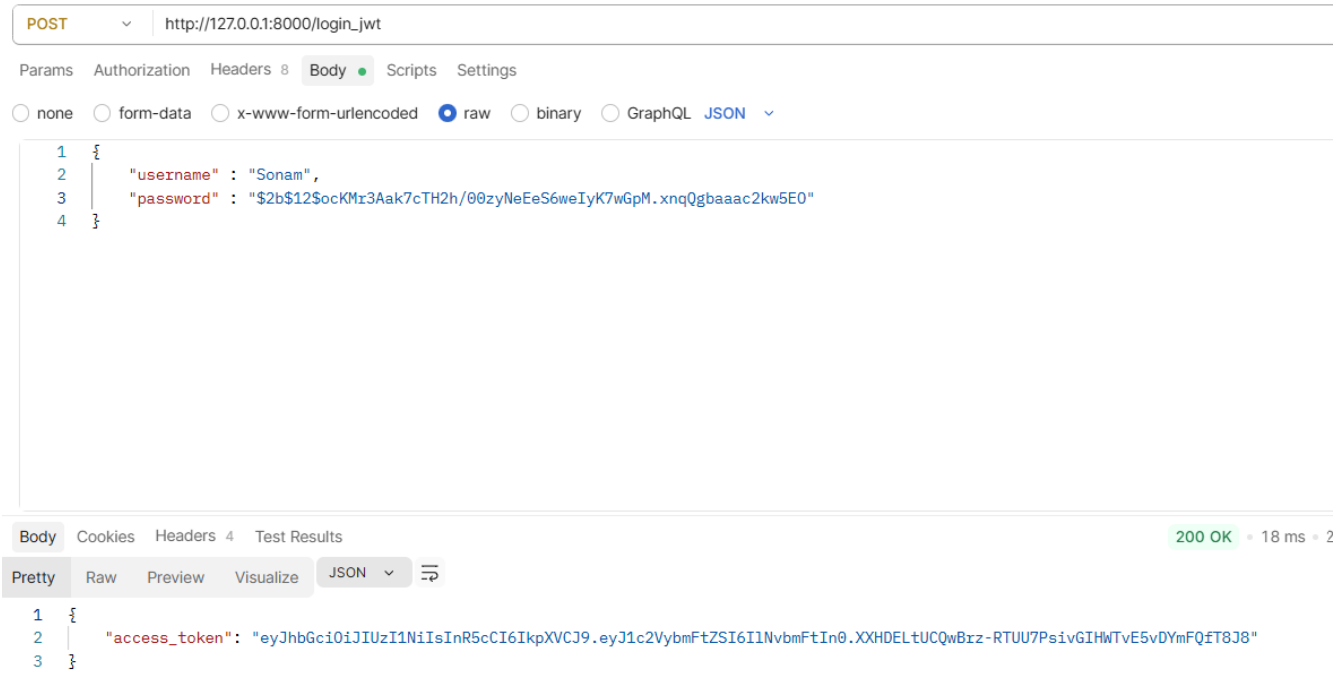
Password is incorrect so showing incorrect password:



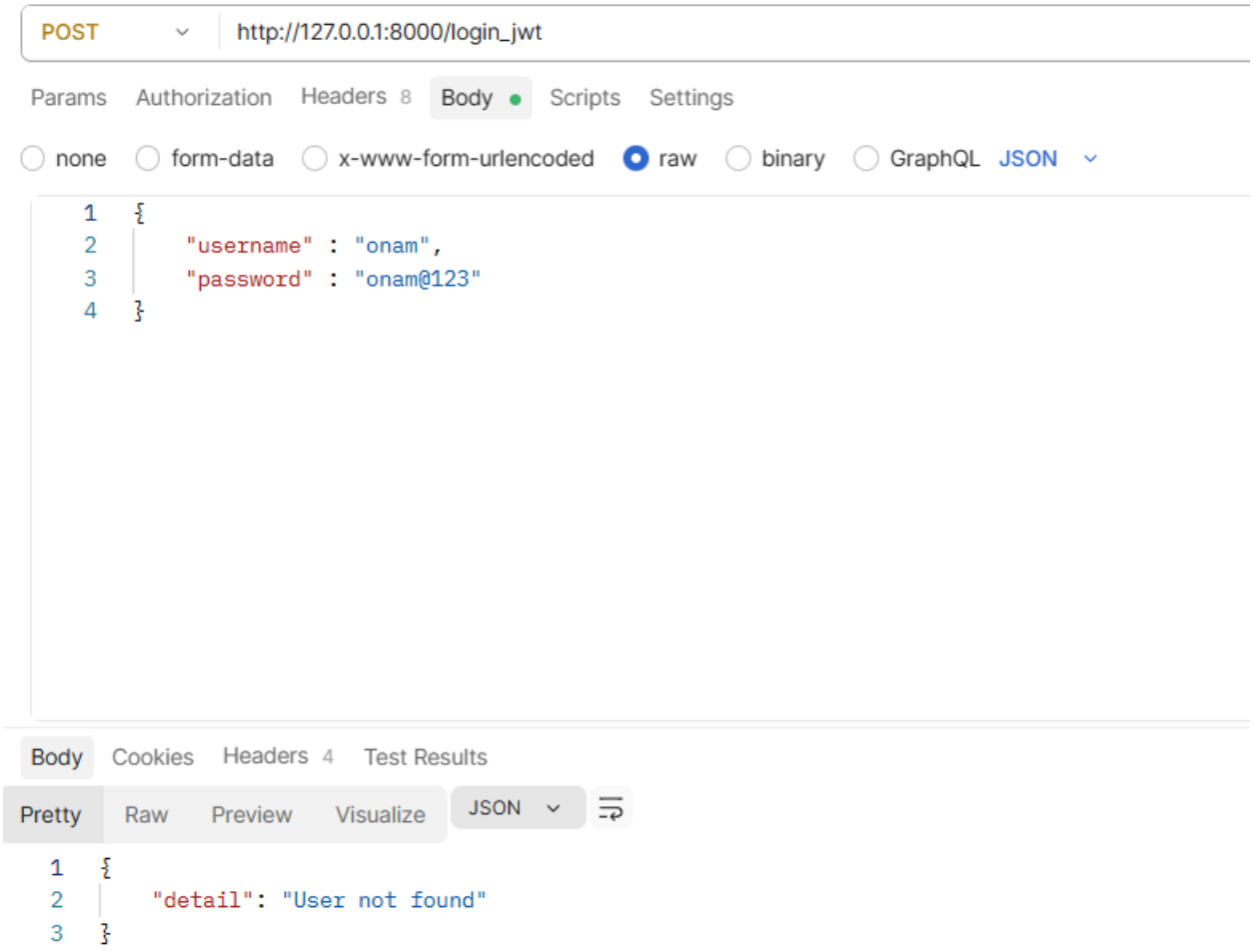
Hashed password cannot fetched by the normal password so showing incorrect password:



Hashed password fetched by the hashed password so it showing access token:



If user doesn’t exists:

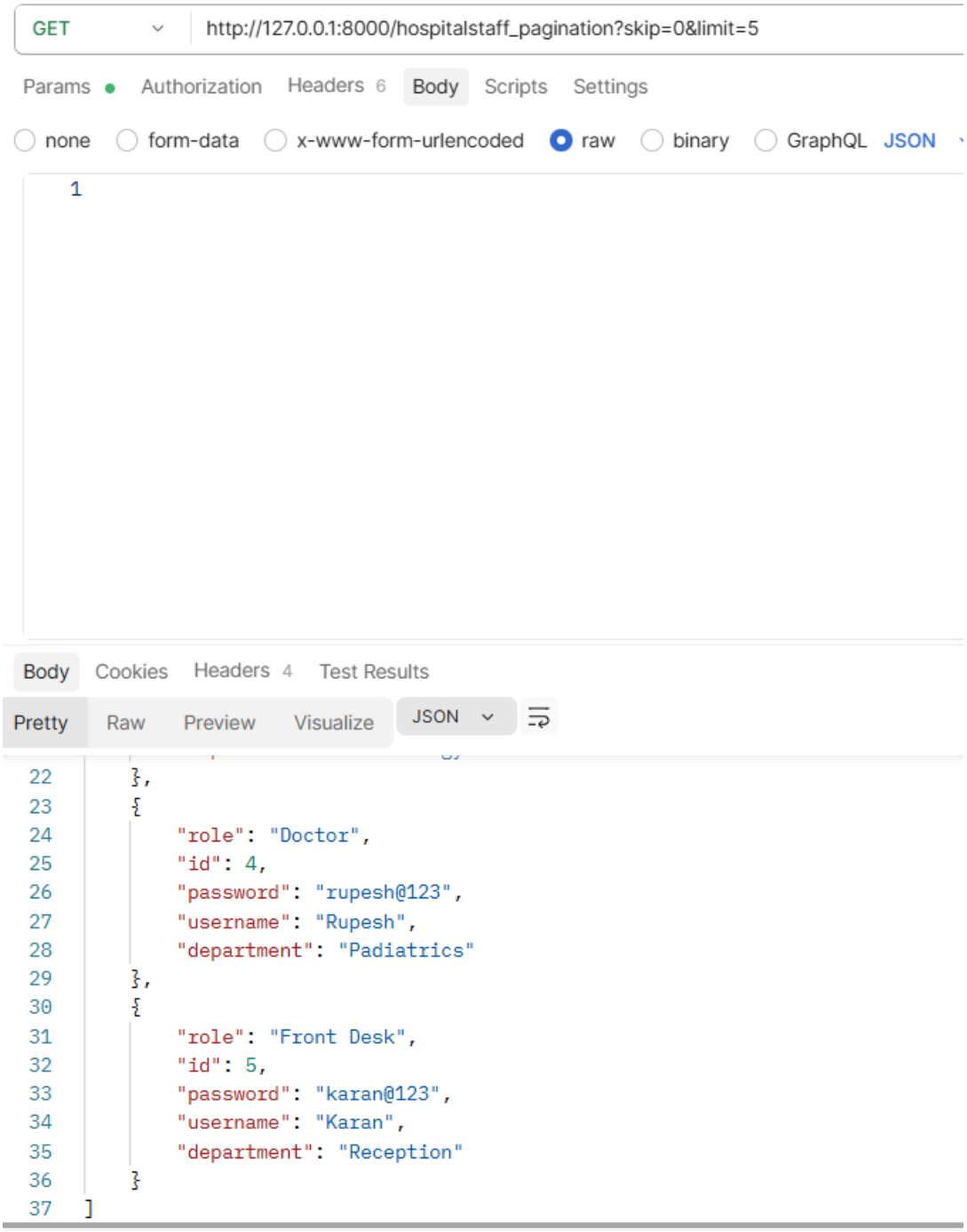


Task 9: Create GET API with Pagination Requirements:

Fetch staff records using:

- **skip**
- **limit Example: /staff?skip=0&limit=5**

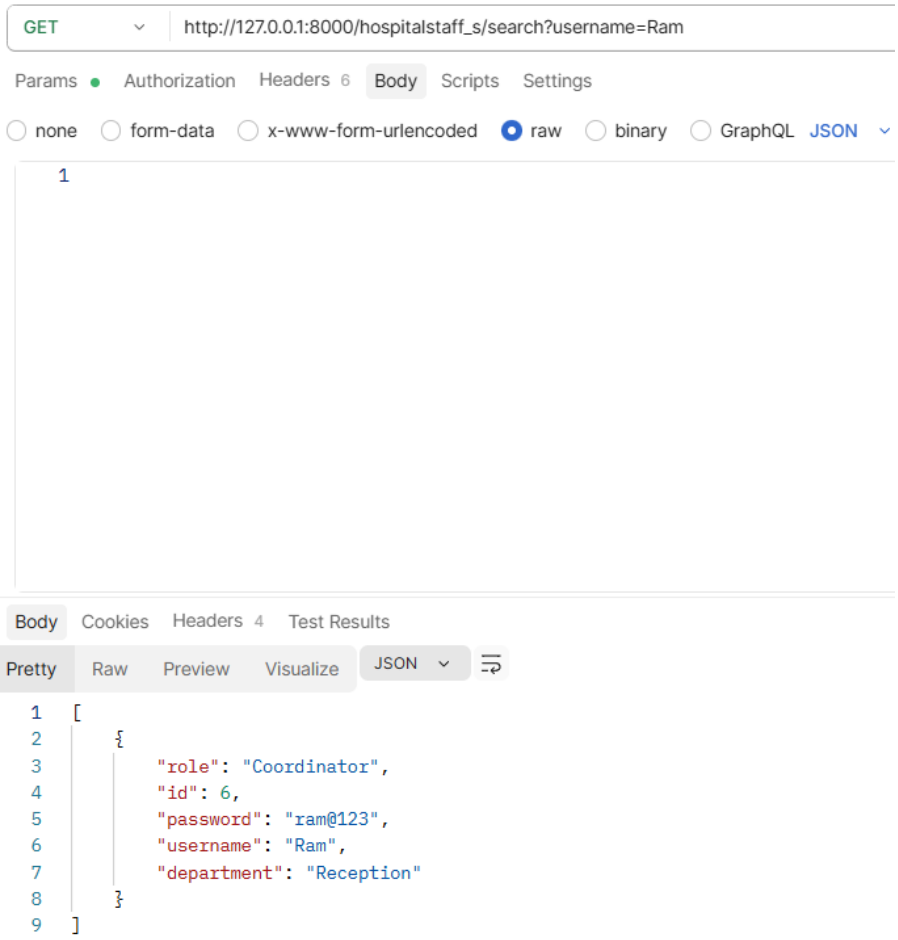
Only first 5 records will be shown:



Task 10: Create Search API Requirements:

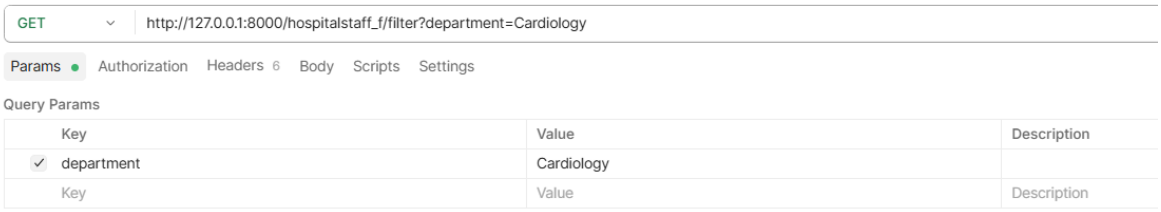
Search staff records by:

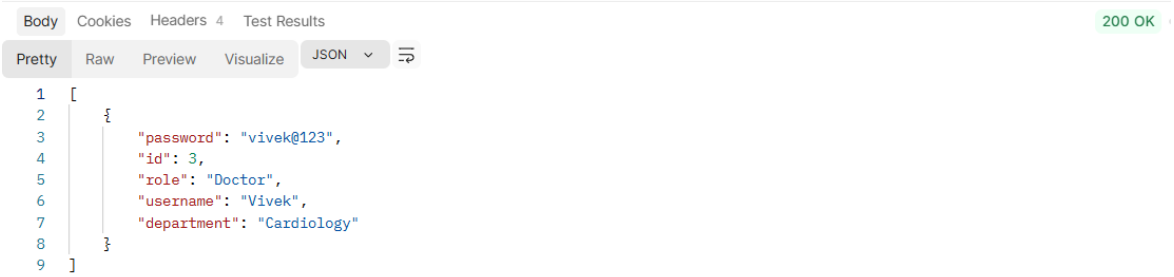
- **username Example: /search?username=amit**



Task 11: Create Filtering API Requirements: Filter staff records by:

• department Example: /filter?department=Cardiology





Task 12: Implement Role-Based Access Control (RBAC) Requirements:

Only Admin can:

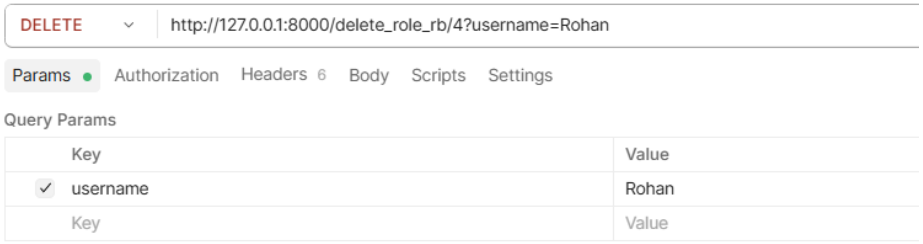
- **Add Staff**
- **Update Staff**
- **Delete Staff**

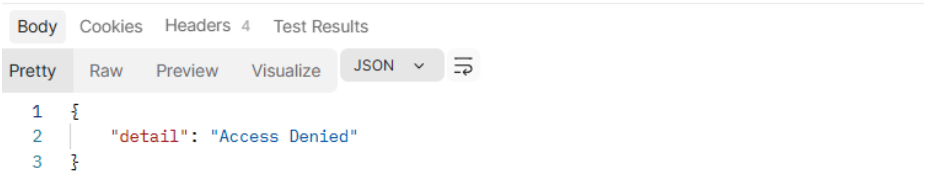
Doctor & Receptionist can:

- **View Staff Records**

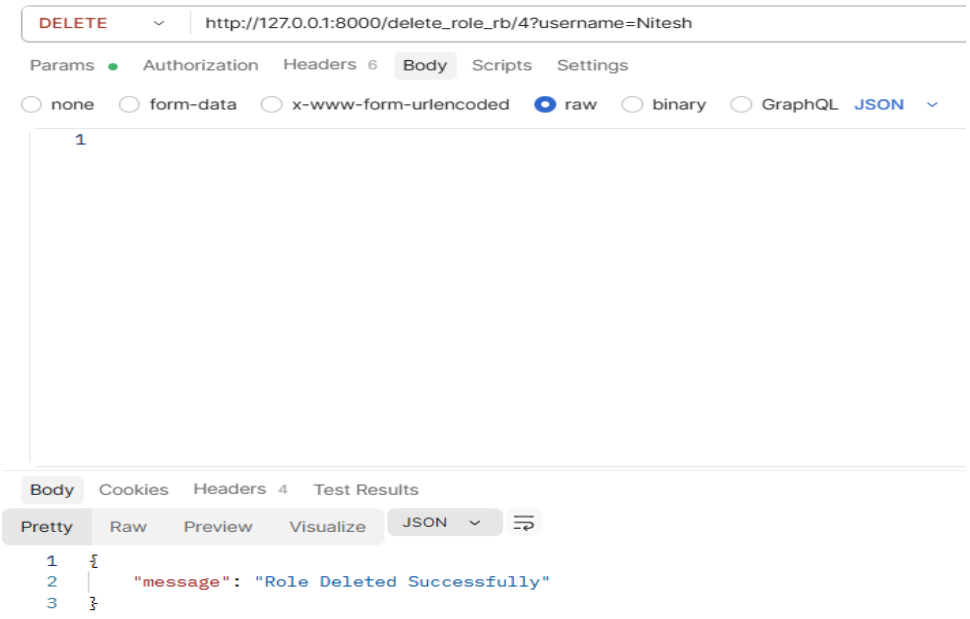
Delete Staff:

Access Denied if the user is not an Admin/Director:



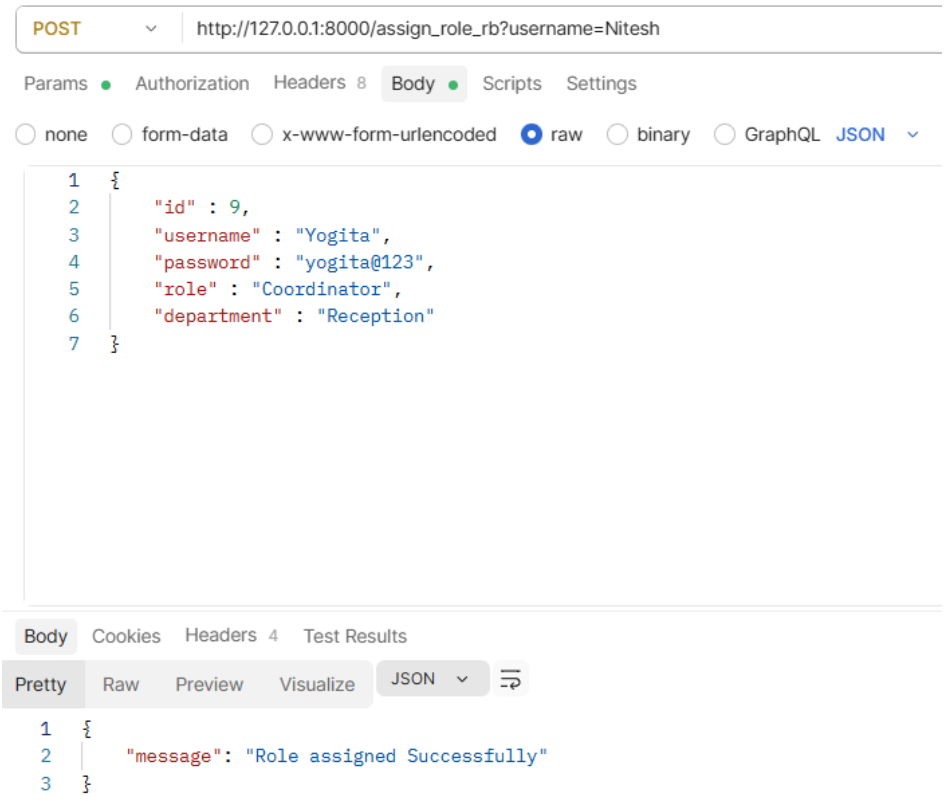


Access Granted if the user is not an Admin/Director:

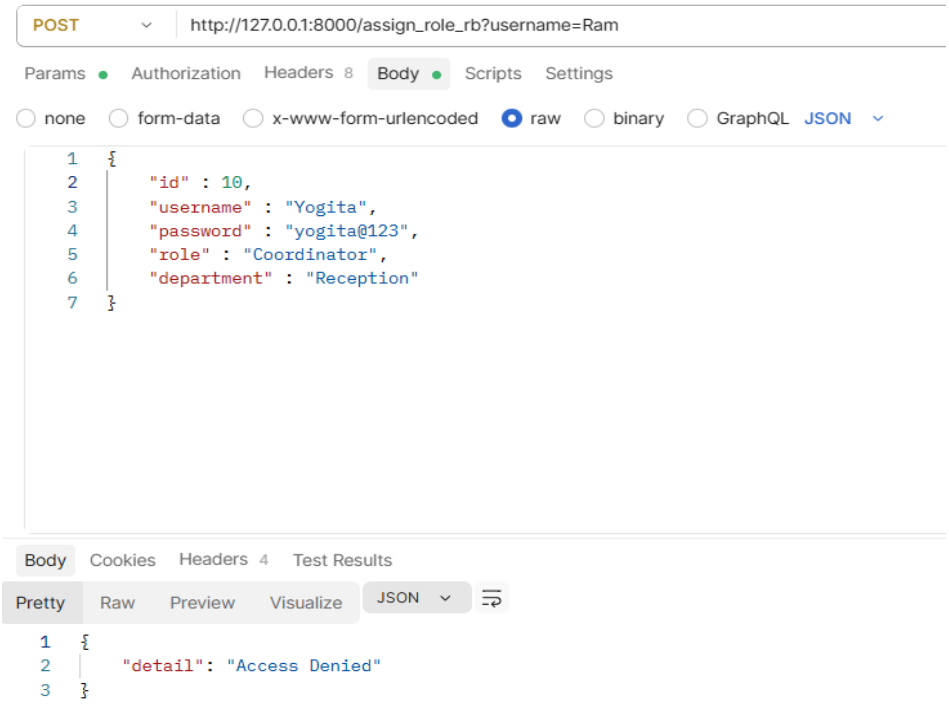


- **Add Staff:**

Access Granted if the user is not an Admin/Director:

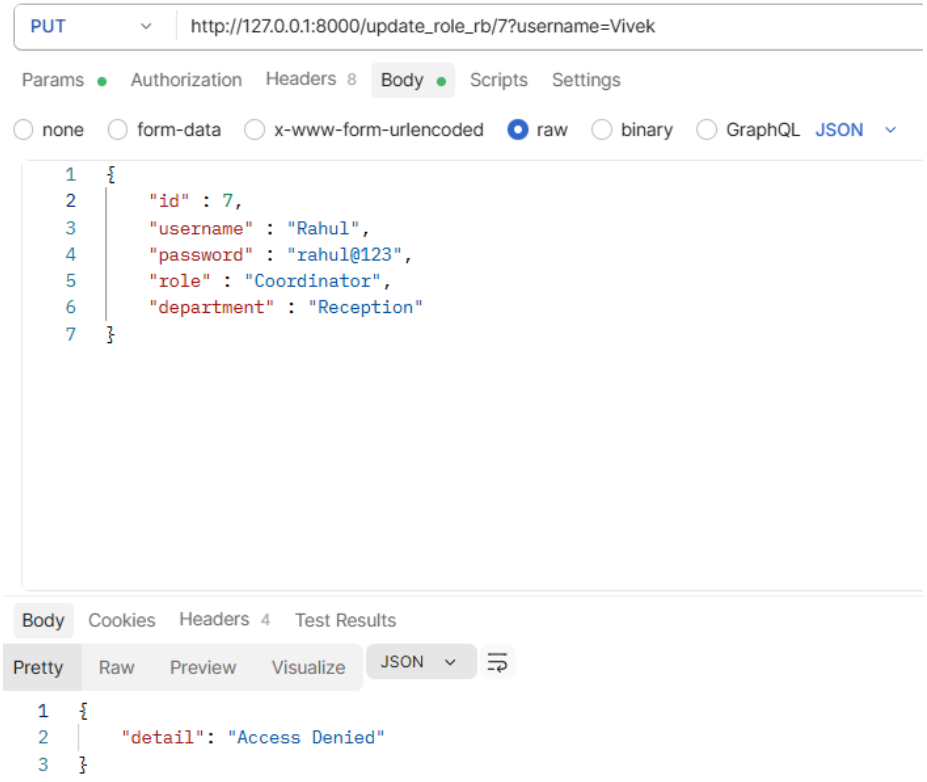


Access Denied if the user is not an Admin/Director:

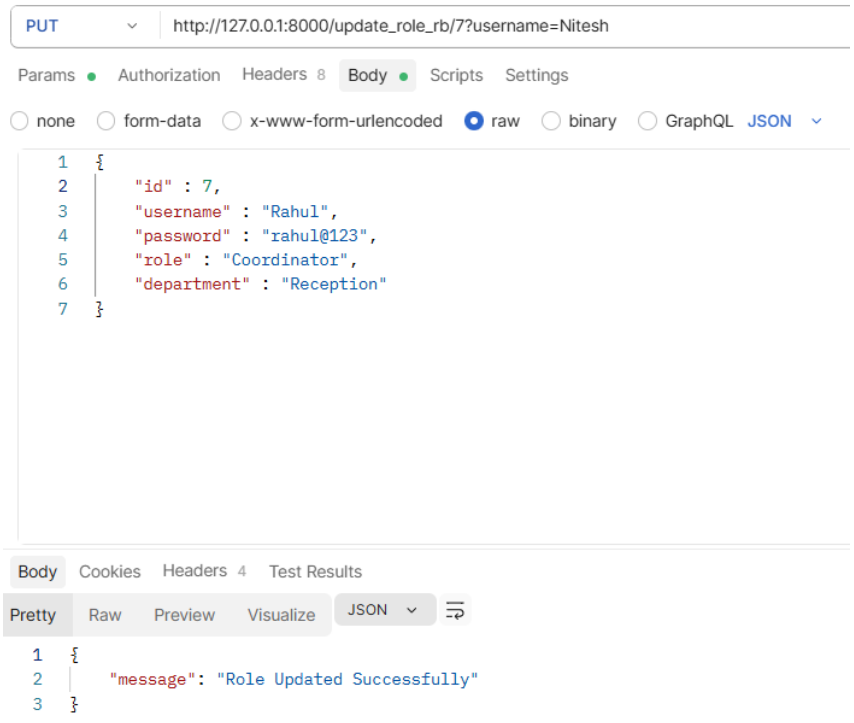


- **Update Staff :**

Access Denied:



Access Granted:



Detailed Database with all above changes:

Data Output Messages Notifications

Showing rows: 1 to 8 Page No: 1 of 1

	id [PK] integer	username character varying	password character varying	role character varying	department character varying
1	1	Nitesh	nitesh@123	Director	Clinical
2	2	Shiksha	shiksha@123	Doctor	Support & Diagnos...
3	3	Vivek	vivek@123	Doctor	Cardiology
4	5	Karan	karan@123	Front Desk	Reception
5	6	Ram	ram@123	Coordinator	Reception
6	7	Rahul	rohan@123	Coordinator	Orthopedics
7	8	Sonam	$2b$12$ocKMr3Aak7cTH2h/00zyNeEeS6welyK7wGpM.xnqQgbaaac2kw...	Doctor	Neurology
8	9	Yogita	yogita@123	Coordinator	Reception

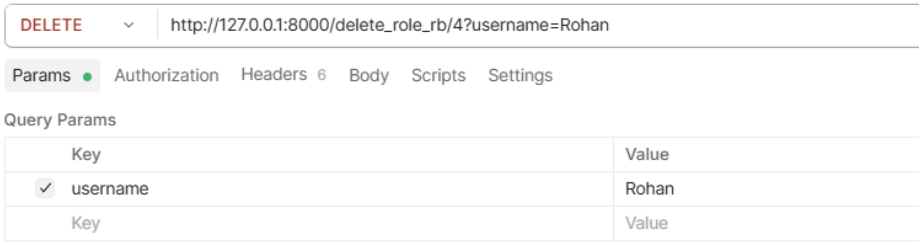
Task 13: Create Admin-Only API Requirements: Protect at least one API using role verification.

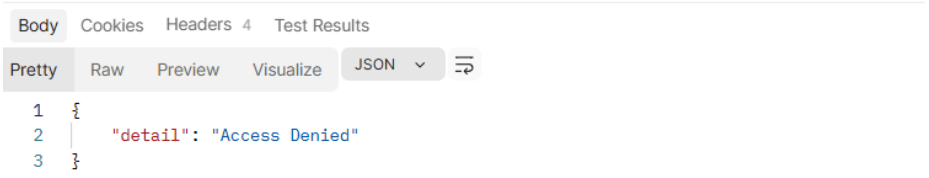
Example:

- **Add Staff OR**
- **Delete Staff**

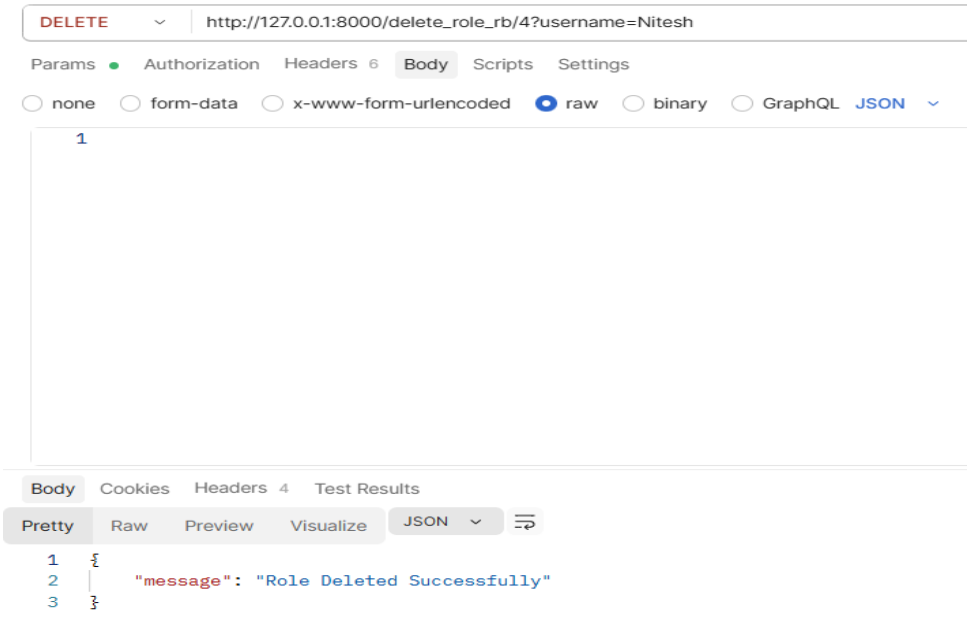
Delete Staff:

Access Denied if the user is not an Admin/Director:



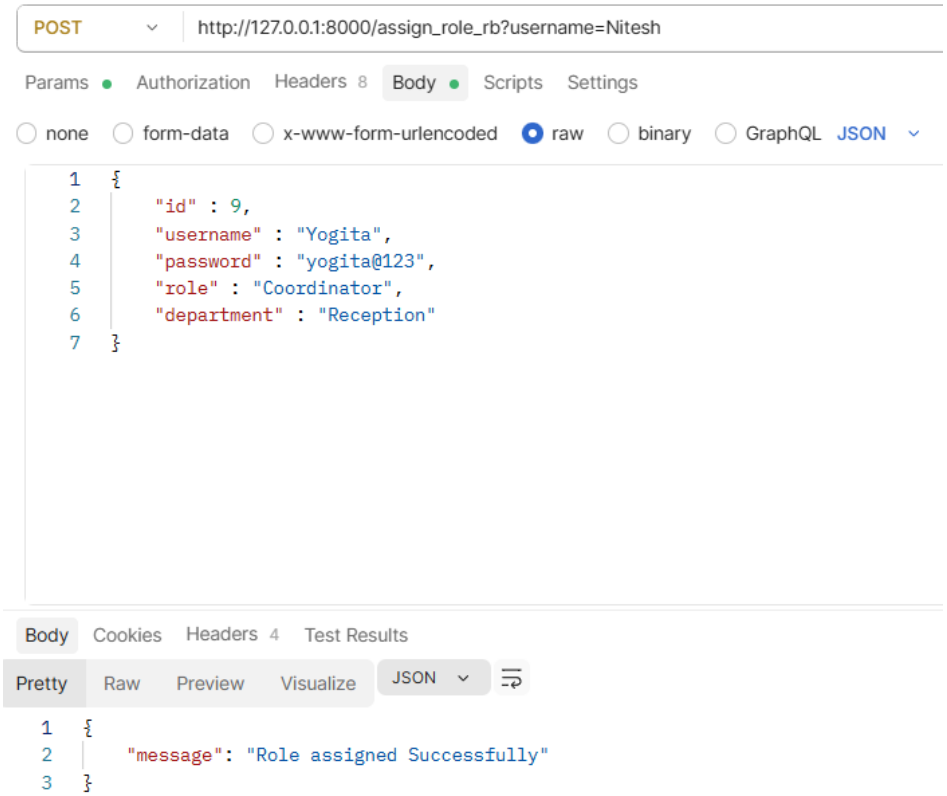


Access Granted if the user is not an Admin/Director:

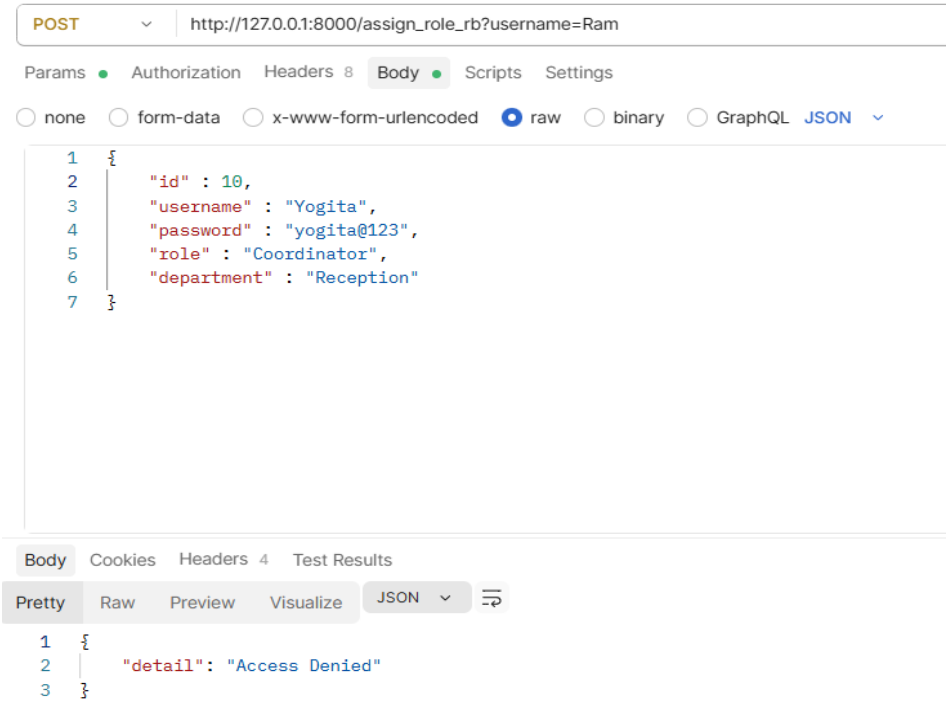


- **Add Staff:**

Access Granted if the user is not an Admin/Director:



Access Denied if the user is not an Admin/Director:



Task 14: Testing Demonstrate:

- **User Registration**
- **Login**
- **JWT Token Generation**
- **Password Hashing**
- **Search API**
- **Filtering API**
- **Pagination API**
- **Role-Based Access Control**

Everything is tested and screenshots added above.